



平顶山学院

PINGDINGSHAN UNIVERSITY

《面向对象程序设计实训》

题 目: SiteLens 网站安全透视系统
的设计与实现

院 (系): 信息工程学院

专业年级: 信息安全 2025 级

组 长: 程 一

成 员:

2026 年 09 月 06 日

摘要

随着互联网应用的快速迭代，现代网站普遍由框架、CDN、统计分析、支付接口等数十种第三方组件拼装而成，任何一个组件的历史漏洞都可能成为整个站点的攻击入口；与此同时，全球漏洞情报以每天数十条的速度增长，仅有效 CVE 编号已超过二十二万条，依靠人工翻查源码、记忆 CVE 的传统评估方式已难以为继。针对这一问题，本系统以信息安全专业的真实工作场景为依托，设计并实现了一个验证型 Web 漏洞扫描器——SiteLens 网站安全透视系统。

系统采用 Python 3.10 语言开发，基于 requests 与 BeautifulSoup 实现网站数据采集与解析，基于 Flask 与 PostgreSQL 实现扫描服务与数据持久化，运用封装、继承、多态、组合等面向对象技术构建了以 BaseDetector 检测器继承体系为核心的扫描引擎。系统实现了目标安全校验、网页证据采集、技术指纹识别（2850 余条指纹、78 个类别）、漏洞主动验证（2613 条社区模板与 42 条自研无害检测项）、漏洞情报三级判定（11024 条情报、OSV 版本区间比较、KEV 在野利用红标）、TLS 与 DNS 网络层检测、源码静态审计、登录爆破授权测试、批量扫描、历史对比与报告导出等功能，并通过零构建的 Web 工作台统一呈现。

系统对 SSRF 攻击、SQL 注入、路径穿越等风险做了针对性防护，全部数据库访问采用字面量 SQL 加参数绑定，扫描行为遵循全局限速、身份标识与 robots 协议，仅面向授权目标使用。经 49 项单元测试与 32 项接口冒烟测试验证，并通过对真实站点的多轮实测，各项功能正确可用，达到了预期的设计目标，具有较强的实用性与工程价值。

关键词：Python；面向对象；网络爬虫；漏洞扫描；信息安全

目 录

1 绪论	1
1.1 背景与意义	1
1.2 设计的主要思路及内容	2
1.3 项目分工	2
2 需求分析	3
2.1 功能需求分析	3
2.2 性能需求分析	5
3 总体设计	6
3.1 系统总体设计	6
3.2 数据库设计	8
4 详细设计与实现	9
4.1 目标校验与扫描引擎主流程	10
4.2 数据采集与解析模块	11
4.3 技术指纹识别模块	12
4.4 验证型检测引擎	13
4.5 漏洞情报关联模块	15
4.6 网络层检测模块	16
4.7 登录爆破与源码审计模块	17
4.8 Web 工作台与数据持久化模块	18
5 系统测试	19
5.1 测试目的	20
5.2 测试用例	20
6 总结	22

1 绪论

1.1 背景与意义

随着互联网应用规模的不断扩大，一个普通网站的背后往往运行着 Web 框架、CDN、统计分析、支付接口、字体库等数十种第三方组件。这些组件由不同社区以不同节奏维护，任何一个组件暴露出的历史漏洞，都可能成为攻击者进入整个站点的入口。以近年发生的安全事件为例，大量入侵的起点并不是目标业务代码本身的缺陷，而是站点引用的某个组件版本停留在已知漏洞的影响区间内。与此同时，全球漏洞情报以每天数十条的速度增长，仅有效 CVE 编号已超过二十二万条，CISAKEV 在野利用清单收录的漏洞也超过一千六百条。传统的网站安全评估依赖人工翻查网页源码、逐个比对响应头，再凭记忆联想相关漏洞，效率低、覆盖面窄、结论难复核，已经无法满足实际工作对时效性与准确性的要求。

本课题面向信息安全专业的真实工作场景，设计并实现了验证型 Web 漏洞扫描器 SiteLens 网站安全透视系统。系统的意义主要体现在三个方面：其一，把「打开开发者工具逐个猜」的手工分析过程，变为一次输入、全自动完成的标准化流程，显著提升了评估效率；其二，将识别结果与漏洞情报按版本区间自动比对，把「同名组件可能有漏洞」的粗放提示升级为「确认受影响 / 可能受影响 / 不受影响」的三级判定，有效抑制了误报；其三，全部扫描行为内置限速、身份标识与 SSRF 防护，仅面向授权目标，符合《中华人民共和国网络安全法》对授权测试的要求，可作为合法授权评估的教学与工程工具。

对本实训而言，该项目覆盖了课程考核的全部核心知识点：扫描引擎以抽象基类与七个检测器子类体现了继承与多态，以实体类私有属性与只读访问器体现了封装，以引擎对采集、解析、匹配等组件的持有关系体现了组合；数据采集环节完整运用了 requests 会话管理与 BeautifulSoup 结构化解析两项爬虫技术；数据库存储、异步任务、进度显示、扫描历史与报告导出等扩展能力则作为综合运用的加分项一并实现。

1.2 设计的主要思路及内容

系统总体思路是「先看清、再验证、后定论」。第一步看清目标：采集首页与同域子页面，从响应头、Cookie、meta 标签、DOM 结构、脚本地址等多类证据中识别网站技术栈；第二步主动验证：按识别出的技术路由社区验证模板与自研无害检测项，用真实请求证实漏洞是否存在；第三步给出结论：将验证结果与漏洞情报按版本区间比对，输出三级判定、安全响应头评分与中文修复建议，并将全部结果持久化以支持检索、对比与导出。三步之间以数据流水线衔接，每一步的产物都是下一步的输入，任一环节的结论都可以回溯到原始证据。

系统采用 B/S 架构与分层设计：展示层为零构建的手写 HTML/CSS/JS 页面；应用层为 Flask 实现的页面路由、REST API 与异步任务队列；核心层为 scanner 包，包含目标校验、采集解析、指纹检测、验证引擎、情报关联、网络层检测、源码审计等子模块；数据层为 PostgreSQL 18 数据库（sitelens 库），存储指纹知识库、漏洞情报与扫描结果。技术栈为 Python 3.10、Flask 3、psycopg2、requests、BeautifulSoup4、PyYAML 与 PostgreSQL 18。

开发过程按「核心引擎—验证能力—外围功能」三阶段推进：第一阶段实现目标校验、采集解析与指纹检测的主链路，打通「输入网址到输出技术清单」的最小闭环；第二阶段引入验证型检测引擎、Nuclei 社区模板转换器与情报三级判定，把系统从「看得清」升级为「验得准」；第三阶段完成 Web 工作台、批量扫描、扫描历史与 diff、情报库检索、报告导出与登录爆破等外围模块，并进行系统测试。每一阶段结束都以单元测试与真实站点实测做回归验证，保证增量开发不破坏已有能力。

1.3 项目分工

本项目由组长统一规划、分模块协作完成，分工如表 1-1 所示。

表 1-1 项目分工

成员	承担工作	占比
组长 程一	总体架构设计；扫描引擎与验证引擎核心编码；数据库模型与 REST API 设计；合规与安全防护方案	40%

成员	承担工作	占比
成员（待填写）	数据采集与解析模块；指纹知识库整理与导入管线；Nuclei 模板转换器	20%
成员（待填写）	Web 前端工作台八个页面；情报库检索页；明暗双主题与交互细节	20%
成员（待填写）	测试用例设计与执行；真实站点实测；文档整理与答辩材料	20%

全部成员共同参与代码走查、合规审查与阶段评审。各模块职责边界清晰，接口由组长统一定义后并行开发，联调阶段每周合并一次并跑全量回归测试。

2 需求分析

系统在立项阶段对照课程考核目标与真实安全评估 workflow，从功能与性能两个维度进行了需求分析，确保每一项功能都有明确的使用场景、输入输出与验收标准。需求分析遵循两个原则：一是所有功能必须落在一个真实的安全评估动作上，不做无法落地的「演示功能」；二是所有主动能力必须可关闭、可审计，把合规当作需求而不是事后补丁。

2.1 功能需求分析

系统功能围绕「一次完整的安全评估」展开，共划分为八个功能模块，整体功能需求如表 2-1 所示。

表 2-1 系统功能需求一览

功能模块	需求描述	对应考核知识点
目标安全校验	对输入 URL 做协议白名单、主机黑名单与 DNS 解析逐 IP 校验，拒绝内网与保留地址	异常处理、字符串处理
数据采集与解析	requests 会话采集页面，BeautifulSoup 解析响应头、Cookie、meta、DOM、脚本等多类证据	requests、BeautifulSoup
技术指纹识别	基于 2850 余条指纹规则识别网站技术栈，输出置信度与命中证据	面向对象（多态）
漏洞主动验证	按指纹结果路由 2613 条社区模板与 42 条自研无害检测项，命中即「已验证漏洞」	面向对象（继承）、文件解析
漏洞情报关联	识别结果关联 11024 条情报，按 OSV 版本区间输出三级判定，KEV 红标	数据库、组合

功能模块	需求描述	对应考核知识点
网络层检测	TLS 证书过期 / 自签 / 弱协议检测，SPF/DMARC/MX 邮件安全审计	socket、DNS 查询
源码审计与登录爆破	16 规则静态审计与 Python 污点分析；授权前提下的登录表单爆破测试	AST 解析、HTTP 会话
工作台与数据管理	Web 工作台、批量扫描、扫描历史与 diff、四格式导出、REST API	Flask、PostgreSQL、进度显示

核心链路上，被动指纹识别、漏洞主动验证与漏洞情报关联三个模块构成递进关系：指纹识别回答「目标是什么」，主动验证回答「疑似漏洞是否真实存在」，情报关联回答「这个漏洞对目标是否构成实际风险」。三者缺一不可——只有指纹会沦为「同名就报漏洞」的误报机器，只有模板验证会缺乏优先级与上下文，只有情报则成了纸上谈兵。

外围模块面向资产管理与授权测试的日常使用：批量扫描支持一次提交最多 50 个站点并实时展示进度；扫描历史支持任意两次扫描的 diff 对比（新增 / 消失 / 版本变化），用于持续监控技术栈变化；情报库提供全库模糊与语义检索；导出覆盖 JSON、明细 CSV、78 列宽表 CSV 与单文件 HTML 报告四种格式，满足入库、报表与归档的不同需要；REST API 全量开放系统能力，便于接入自动化流水线。

以一个典型使用场景说明功能需求：安全人员输入授权目标网址，选择「深度识别」等级，系统依次完成目标校验、证据采集、指纹识别、模板验证与情报关联，页面实时显示当前阶段与进度；扫描完成后给出技术清单（含置信度）、已验证漏洞、情报三级判定、安全响应头评分与修复建议；人员可将结果导出为宽表 CSV 归档，或留存至历史供下次对比。



图 2-1 系统首页（输入入口与实时统计）

2.2 性能需求分析

性能需求围绕「合规优先、够用高效」制定，各项指标如表 2-2 所示。

表 2-2 性能需求与实测结果

指标	需求	实测结果
访问礼貌性	全局请求间隔 $\geq 0.4s$, UA 自报身份, 遵循 robots.txt	达标 (RateLimiter 全局生效)
并发能力	同时扫描任务 ≤ 3 , 超出排队并实时显示进度	达标 (信号量 + jobs 队列)
快速扫描耗时	单站 $\leq 15s$	实测 约 5.7s (text-well.com 标准识别)
全面扫描耗时	含模板验证 $\leq 90s$	实测 约 59s (text-well.com)
指纹匹配正确性	短词不误报 (如 plex 不得命中 complex)	达标 (非字母数字边界匹配)
验证误报控制	模板命中须二次确认, 软 404 不误报	达标 (基线比对 + 重放确认)
数据安全	SQL 全参数绑定, 上传源码审计后即删	达标 (静态安全审计通过)

其中访问礼貌性是安全工具的第一约束: 扫描器对目标而言是一个「高频率的陌生访客」, 如果不加约束, 极易对小型站点造成事实上的压力测试, 也会触发防护设

备封禁。因此系统把限速做成了全局强制项而非可选配置，任何模块发起的请求都必须经过同一个限速器。数据安全方面，系统全部数据库访问采用字面量 SQL 加 %s 参数绑定，从编码层面杜绝拼接注入；源码审计功能接收的用户上传内容在临时目录中处理，审计结束立即删除，不在服务器留下任何副本。

3 总体设计

本章从系统分层、面向对象结构与数据库三个方面说明总体设计。

3.1 系统总体设计

系统采用四层架构，如图 3-1 所示。展示层与应用层之间以 fetch/JSON 交互，应用层与核心层之间通过异步任务队列解耦：扫描请求入队后由后台线程执行，前端轮询任务进度，避免了长请求阻塞。核心层按职责划分为八个高内聚模块，全部通过显式接口与数据类交互，不共享可变全局状态。

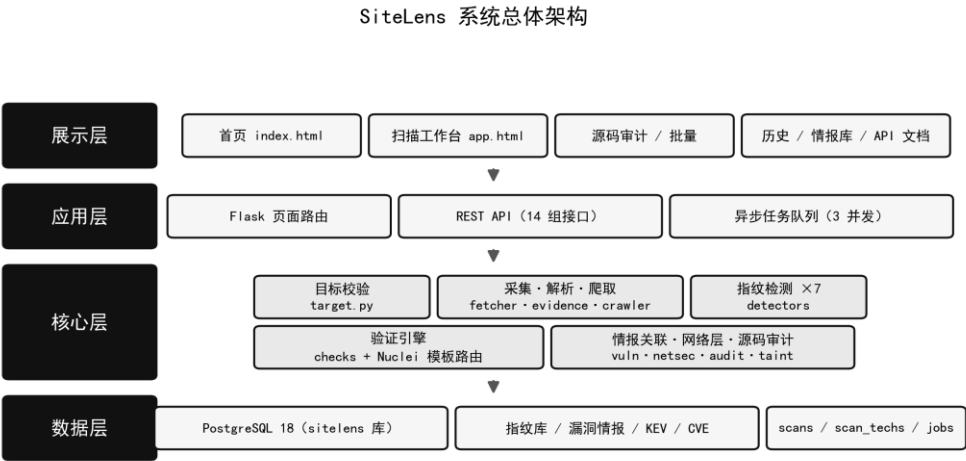


图 3-1 系统总体架构

表 3-1 系统分层职责

层次	组成	职责
展示层	index / app / audit / batch / history / intel / api-docs / about 八页	输入交互、进度渲染、结果可视化，零构建无框架
应用层	app.py: 页面路由 + REST API + 任务队列	请求调度、参数校验、异步执行、Token 鉴权
核心层	scanner 包: target / fetcher / evidence / crawler / detectors / checks / vuln / netsec / audit / taint	扫描链路全部业务逻辑，纯库无界面
数据层	PostgreSQL 18 (sitelens 库) 9 张表	知识库与扫描结果持久化，支持检索与统计

面向对象结构的核心是 ScannerEngine 组合根与 BaseDetector 继承体系，如图 3-2 所示。引擎在构造时持有采集器、爬虫、指纹库索引与情报匹配器四个组件（组合关系），扫描时对检测器列表统一调用 detect() 接口（多态），不区分具体类型；新增检测器只需继承基类并注册一行，引擎零改动。实体类（ScanTarget、PageEvidence、ScanResult、SecurityReport 等）全部采用双下划线私有属性加只读 property 对外暴露，仅提供受控写入点——例如 PageEvidence 仅允许解析器回填一次标题，Technology 的置信度在新增独立证据来源时按规则自动提升——保证扫描过程中的数据不可被外部随意篡改。四大面向对象特性的落点汇总见表 3-2。

核心类关系图（封装 / 继承 / 多态 / 组合）

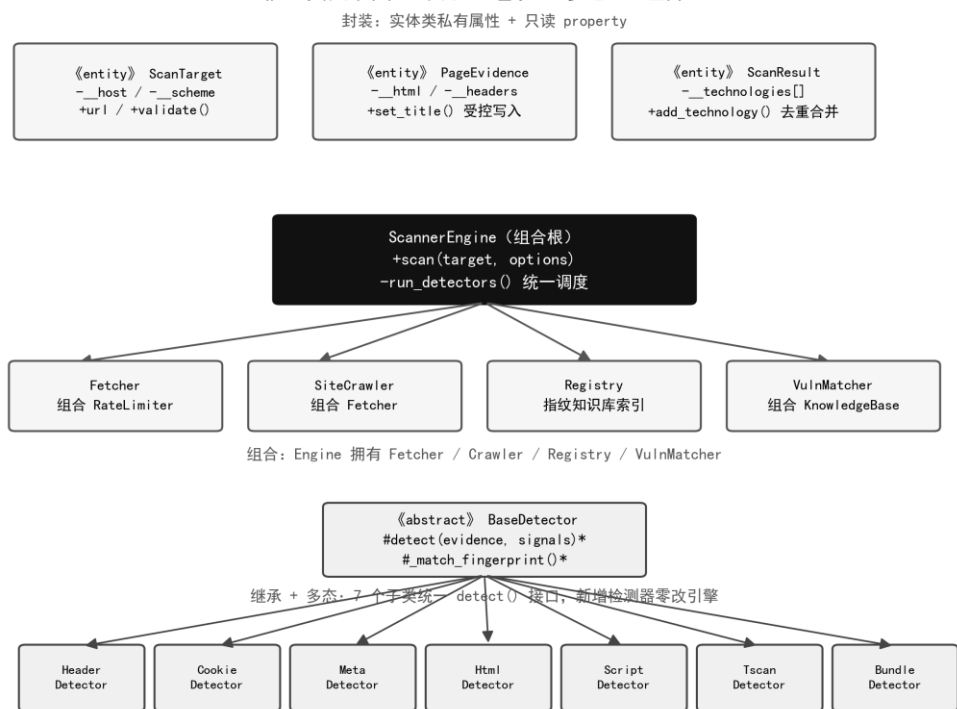


图 3-2 核心类关系图

表 3-2 面向对象四大特性的实现落点

面向对象特性	实现位置	设计要点
封装	models.py 全部实体类	双下划线私有属性 + 只读 property；仅 set_title()、add_evidence() 两个受控写入点
继承	detectors/base.py → 7 个子类	BaseDetector 抽象模板方法 detect()，子类各消费一类证据
多态	engine._run_detectors()	对检测器列表统一调用接口，新增检测器引擎零改动
组合	ScannerEngine 及各组件	Engine 拥有 Fetcher/Crawler/Registry/VulnMatcher；Fetcher 内组合 RateLimiter

这种结构带来的直接收益是扩展成本极低：实训后期相继引入表达式指纹检测器、bundle 签名检测器与验证引擎，均未修改引擎主流程；数据驱动的指纹规则（数据库行而非硬编码）则让指纹库扩充完全脱离代码发布。

3.2 数据库设计

系统使用 PostgreSQL 18 存储知识库与扫描数据，数据库名为 sitelens，共设计九张表，如表 3-3 所示。知识库表与结果表分离：categories、technologies、tscan_fingerprints、service_fp、vuln_kb、cve_ms、kev 为静态或定期更新的知识数据；scans、scan_techs、jobs 为扫描产生的业务数据。

表 3-3 数据库主要数据表

表名	用途	规模（实测）
categories	78 个检测类别字典(ord 列即宽表 CSV 列序)	78 条
technologies	自建精编指纹（rules JSONB：headers/cookies/meta/html/dom/scripts）	370 条
tscan_fingerprints	社区表达式指纹（groups JSONB：组内 AND、组间 OR）	2481 条
service_fp	nmap 风格端口服务指纹(pattern 预验证可编译)	11966 条
vuln_kb	漏洞情报库（product trgm 索引 + affected 版本区间 + 256 维语义向量）	11024 条
cve_ms	微软安全公告索引（component 索引）	34931 行

表名	用途	规模（实测）
kev	CISA 在野利用清单	1695 条
scans / scan_techs	扫描主表（JSONB 全量）+ 技术明细（关系型可检索）	随使用增长
jobs	异步扫描/爆破任务队列（状态、进度、结果）	随使用增长

以核心的漏洞情报表 vuln_kb 为例，其字段设计如表 3-4 所示。product 字段建立 trgm 模糊索引（GIN）支持亚秒级相似检索；affected 字段以「|」分隔存储多组 OSV 版本区间；embedding 字段存储 256 维哈希 TF-IDF 向量用于语义排序。扫描结果主表 scans 以 JSONB 保存全量结果保证完备性，明细表 scan_techs 抽出「站点-技术」关系支持反向检索。

表 3-4 vuln_kb 漏洞情报表主要字段

字段	类型	说明
id / cve	TEXT	情报唯一标识与关联 CVE 编号
product / name	TEXT	受影响产品名（trgm 索引）与漏洞名称
severity	TEXT	critical / high / medium / low / info
affected	TEXT	OSV 版本区间，多组以「 」分隔，如 >=8.3,<8.3.7 2.x
cvss_v3 / embedding	TEXT / float8[]	CVSS 评分与 256 维语义向量
src / ref	TEXT	来源（afrog/xray/tscan/ghsa/osv）与参考链接

全部数据库访问统一封装在 storage 层，采用字面量 SQL 加 %s 参数绑定，静态安全扫描通过；模糊检索使用 position()/similarity() 函数而非字符串拼接，动态排序字段以白名单映射，不存在任何注入面。

4 详细设计与实现

本章按扫描链路顺序展开各模块的实现。系统一次完整扫描的主流程如图 4-1 所示，共八个阶段，全部阶段进度实时上报前端。

扫描引擎主流程



图 4-1 扫描引擎主流程

4.1 目标校验与扫描引擎主流程

目标校验是扫描前的安全闸门。ScanTarget 类对用户输入依次执行三项检查：协议白名单（仅允许 http/https）、主机名黑名单（拒绝 localhost、*.local、*.internal 与云元数据地址）、DNS 解析后逐 IP 校验（拒绝环回、私有、链路本地、保留与组播地址）。逐 IP 校验使得借助 DNS 重绑定绕过黑名单的攻击方式失效——即便域名首次解析返回公网地址，扫描器仍会在每次请求前校验实际连接的 IP，工具自身不会被利用为访问内网的跳板。校验不通过时抛出 TargetError，由应用层转换为明确的中文错误提示返回前端。

主流程由 ScannerEngine.scan() 编排，各阶段通过 progress 回调实时上报「已完成/总数 + 当前动作」，前端据此渲染进度条与阶段面板；任一阶段抛出的异常都被捕获并转换为任务错误状态，不会导致服务崩溃。扫描过程中产生的所有中间数据（证据对象、命中记录、评分）都以只读视图暴露给上层，保证结果不可被扫描

流程之外的代码篡改。

4.2 数据采集与解析模块

采集模块 `Fetcher` 封装 `requests.Session`，核心是「全局唯一」的限速器：无论哪个模块发起请求，都必须经过同一个 `RateLimiter`，保证对任意目标的请求间隔不低于 0.4 秒。其实现只在进程内维护上次请求时间戳，每次请求前计算需要等待的剩余时间：

```
class RateLimiter:
    def __init__(self, interval):
        self.__interval = interval    # 最小请求间隔（秒）
        self.__last = 0.0            # 上次请求时间戳

    def wait(self):
        elapsed = time.time() - self.__last
        if elapsed < self.__interval:
            time.sleep(self.__interval - elapsed)
        self.__last = time.time()
```

解析模块基于 `BeautifulSoup` 将 HTML 拆解为结构化证据对象 `PageEvidence`，包括 meta 标签、脚本地址与内联脚本、样式表、CSS 类名集合、DOM 选择器命中池与站内链接清单；favicon 图标会计算 mmh3 哈希用于 CDN 识别。深度模式下爬虫 `SiteCrawler` 以目标 URL 为起点做同域广度优先爬取，最多 4 页，爬取前先解析 robots.txt 并跳过 Disallow 路径；多页证据合并后送入检测器，显著提升了仅在子页面露出的组件（如统计脚本、客服组件）的检出率。

扫描工作台界面如图 4-2 所示：左侧为扫描配置（目标、等级、认证会话、可选 UA），右侧主区在空闲态给出引导，扫描开始后切换为实时阶段面板与进度条。



图 4-2 扫描工作台界面

4.3 技术指纹识别模块

指纹识别采用「证据驱动 + 规则数据化」的设计。指纹规则全部存于数据库：370 条自建精编指纹以 JSONB 描述 headers/cookies/meta/html/dom/scripts 六类匹配条件；2481 条社区指纹则以表达式组存储（组内 AND、组间 OR，支持 != 取反），导入期预解析为结构化组，运行期仅做子串与边界匹配，单页全量匹配耗时低于 1 秒。

七个检测器按证据类型分工：Header、Cookie、Meta、Html、Script 五个检测器分别消费对应证据字段，TscanDetector 独立解释表达式组，BundleDetector 抓取同域主 bundle 文件做库签名匹配，补齐「打包进产物后消失」的前端框架。识别结果经 add_technology() 去重合并：同一技术的多个独立证据来源会合并为一条记录，置信度随来源数提升（每来源 +5，上限 100），版本号从响应头、generator 标签与脚本文件名中用正则捕获。

以对真实站点 text-well.com 的实测为例：深度识别共检出 13 项技术，按类别分组展示（内容分发网络 Cloudflare、托管平台 Vercel、性能类 HTTP/3 与 Cloudflare Analytics、SEO 类 Open Graph / Twitter Cards / schema.org 等），每条均可展开查看命中证据；整站识别耗时约 5.7 秒，结果如图 4-3 所示。

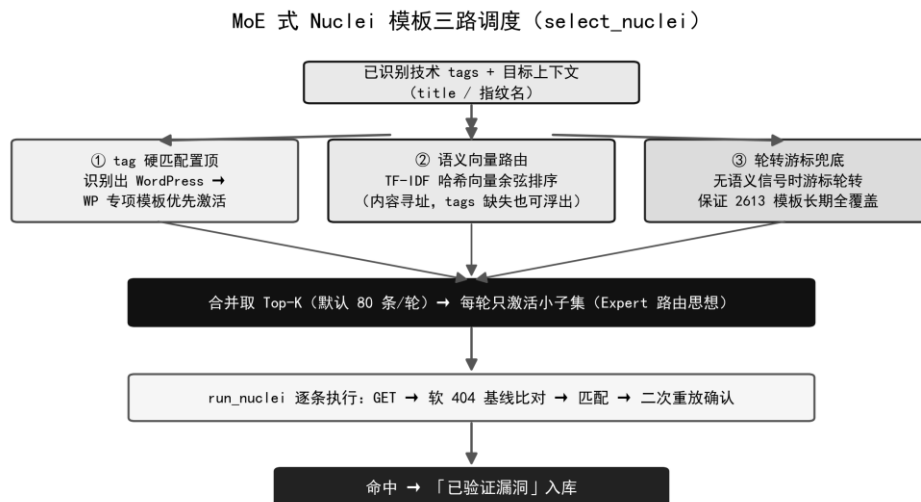


图 4-4 MoE 式 Nuclei 模板三路调度

三路调度的核心实现如下：tag 命中的模板直接进入优先队列；其余模板与目标上下文（标题 + 指纹名）计算余弦相似度排序；无语义信号时按文件游标轮转，保证模板库长期全覆盖。

```

def select_nuclei(cap=80, tech_tags=(), query_text=""):
    rows, vecs = load_nuclei(), _nuclei_vectors(load_nuclei())
    tags = {t.lower().strip() for t in tech_tags if t}
    hit, rest = [], []
    qvec = embed_text(query_text) if query_text else None
    for r, vec in zip(rows, vecs):
        if tags & {t.lower() for t in r.get("tags", [])}:
            hit.append(r)  # ① 指纹 tag 置顶
        else:
            sim = cosine(qvec, vec) if qvec else 0.0
            rest.append((_SEV_ORDER[r["sev"]] - sim * 10, r))
    rest.sort(key=lambda x: x[0])  # ② 语义相似度排序
    rest = rotate_by_cursor(rest, cap)  # ③ 游标轮转兜底
    return (hit + rest)[:cap]
  
```

执行阶段每条模板只允许请求站内相对路径（防止伪造目标），先请求不存在的随机路径建立软 404 基线，与基线相同的响应直接跳过；命中后再立即重放一次，两次均命中才采信，以排除 CDN 多节点响应抖动造成的假阳性。命中记录统一升级为「已验证漏洞」，与情报提示分区呈现。匹配判定逻辑如下：

```

def _group_match(group, status, body_l, header_l):
    if group.get("s") and status not in group["s"]:  # 状态码
        return False
    for w in group.get("wall", []):  # 组内 AND 关键词
        if w not in body_l: return False
    if group.get("wany") and not any(w in body_l  # 组间 OR 关键词
        for w in group["wany"]): return False
  
```

```

for n in group.get("n", []):
    if n in body_l: return False
return True
# 排除词（防误报）

```

此外引擎还包含轻量 DAST 检测（反射 XSS 标记回显、报错型 SQL 注入、开放重定向、目录遍历）与认证扫描能力：用户可注入 Cookie 会话使爬取与检测覆盖登录后才可见的页面。

4.5 漏洞情报关联模块

情报模块 VulnMatcher 将识别结果映射到已知漏洞，三级判定流程如图 4-5。系统内置 11024 条漏洞情报（afrog/xray/TscanPlus 的 POC 元数据，载荷不落盘不执行）、1323 条带 OSV 版本区间的结构化记录与 1695 条 CISA KEV 在野利用清单。判定规则：当目标组件带精确版本号且存在区间数据时，仅输出「确认受影响」结论；同名但无版本号输出「可能受影响」提示；版本落在区间之外直接过滤。命中 KEV 清单的情报附加红标，提示该漏洞已被在野利用、应优先处置。

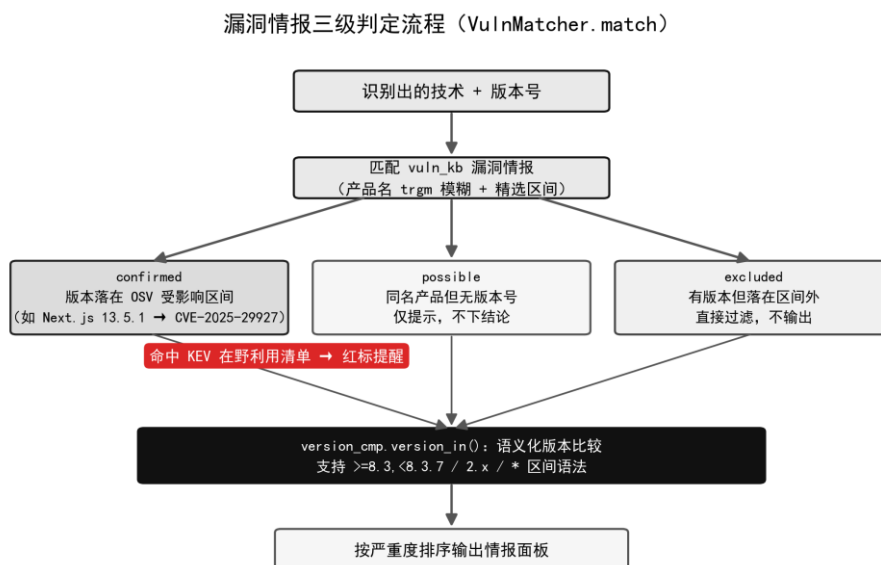


图 4-5 漏洞情报三级判定流程

版本比较器支持 ≥ 8.3 , $< 8.3.7$, $2.x$, $*$ 等区间语法，可表达 OSV 数据的全部区间形态；情报检索采用 trgm 相似度粗筛 (GIN 索引) 加向量余弦重排的混合方案。情报库检索页支持按产品名、关键词实时检索，如图 4-6 所示。

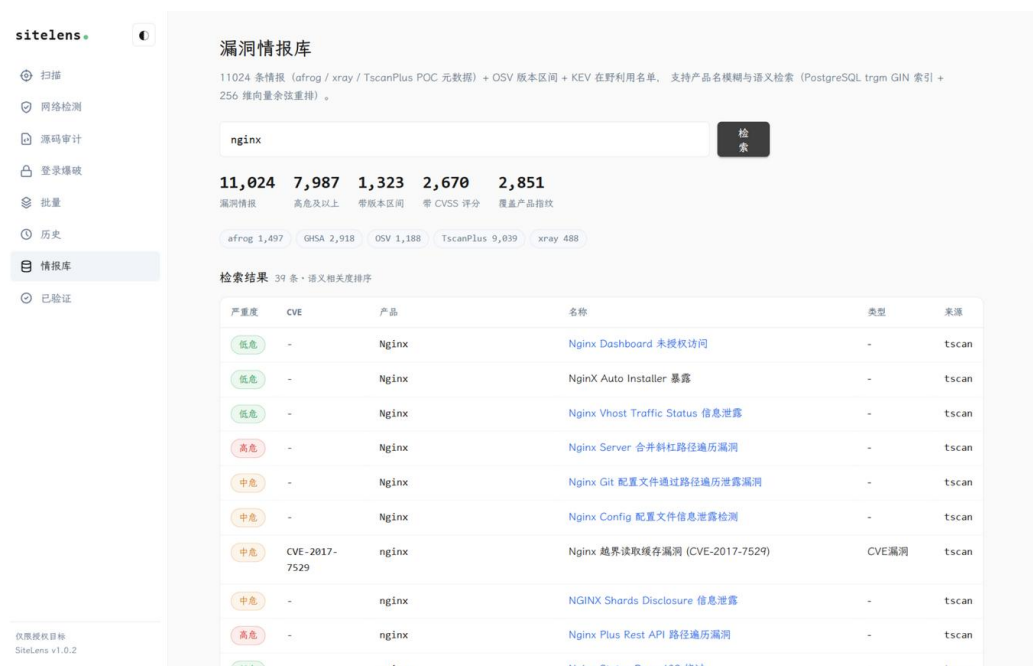


图 4-6 漏洞情报库检索

4.6 网络层检测模块

网络层检测从传输层与域名体系两个视角补充 Web 层看不到的问题。TLS 检测项包括：证书链是否过期或 30 天内临期、是否自签名、证书域名与目标是否匹配、是否仍协商 SSLv3/TLS1.0 等旧协议；邮件安全检测通过 nslookup 查询 SPF、DMARC、MX 记录，判断域名是否可被伪造发件，检测前先将子域名归位到组织主域（如 send.example.com 的 SPF 问题应归并到 example.com 评估），避免误报；端口服务识别对 6 个常见 Web 端口做 TCP 连接，先被动读取 banner，静默则发送最小 HTTP 探测，用 11966 条 nmap 风格指纹匹配服务与版本。全部检测均为只读操作，默认关闭、显式开启。界面如图 4-7 所示。



图 4-7 网络层检测界面

4.7 登录爆破与源码审计模块

登录爆破面向授权测试场景：先用 BeautifulSoup 解析登录页表单（兼容无 form 标签、字段无 name、中文 placeholder 等情况），自动识别验证码字段，随后按用户名-密码字典组合提交，以失败基线加响应差异判定命中，总请求数硬性封顶 300 次；每次尝试前重新加载登录页以刷新验证码，配合本地 OCR 实现数字/运算/点选三类验证码的分级处理。模块入口设置了授权确认门禁，未勾选授权声明无法启动。

源码审计模块支持上传源码文件或压缩包，16 条危险规则覆盖命令执行、反序列化、任意文件读取等模式；污点分析基于 ast 模块追踪「不可信输入到危险执行点」的同函数赋值链——把 request.args 等用户输入标记为 source，把危险执行函数标记为 sink，赋值与字符串拼接都会传播污点标记，比单行正则更接近真实数据流。审计在临时目录完成，结束后立即删除全部上传内容。界面如图 4-8 所示。

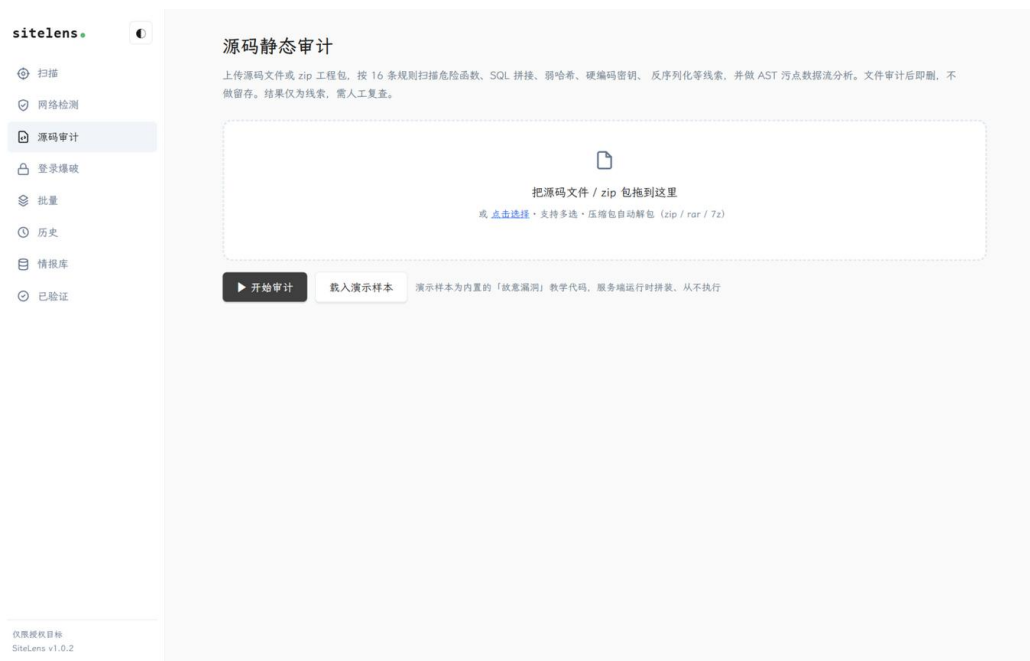


图 4-8 源码静态审计界面

4.8 Web 工作台与数据持久化模块

Web 工作台采用零构建的手写 HTML/CSS/JS 实现, 明暗双主题, 包含首页、扫描工作台、源码审计、批量扫描、扫描历史、情报库检索、API 文档与产品介绍八个页面。扫描以异步任务执行 (3 并发信号量), 前端轮询 `/api/job/<id>` 渲染实时阶段面板; 结果页按「概要—技术分组—已验证漏洞—情报提示—安全评分」分区展示, 每条结论可展开原始证据。

持久化由 ScanStore/JobStore 封装: 扫描主表存 JSONB 全量结果, 技术明细表建立关系型索引以支持「哪些站点使用了某技术」的检索; 历史页列出全部扫描记录, 勾选两次即可 diff 出新增、消失与版本变化的组件, 点击任一记录可就地展开详情 (概要、指纹、情报、评分), 如图 4-9 所示; 结果支持 JSON、明细 CSV、78 列宽表 CSV 与单文件 HTML 报告四种导出格式。对外 REST API 使用 Token 鉴权, 全部接口在 API 文档页集中说明, 如图 4-10 所示。

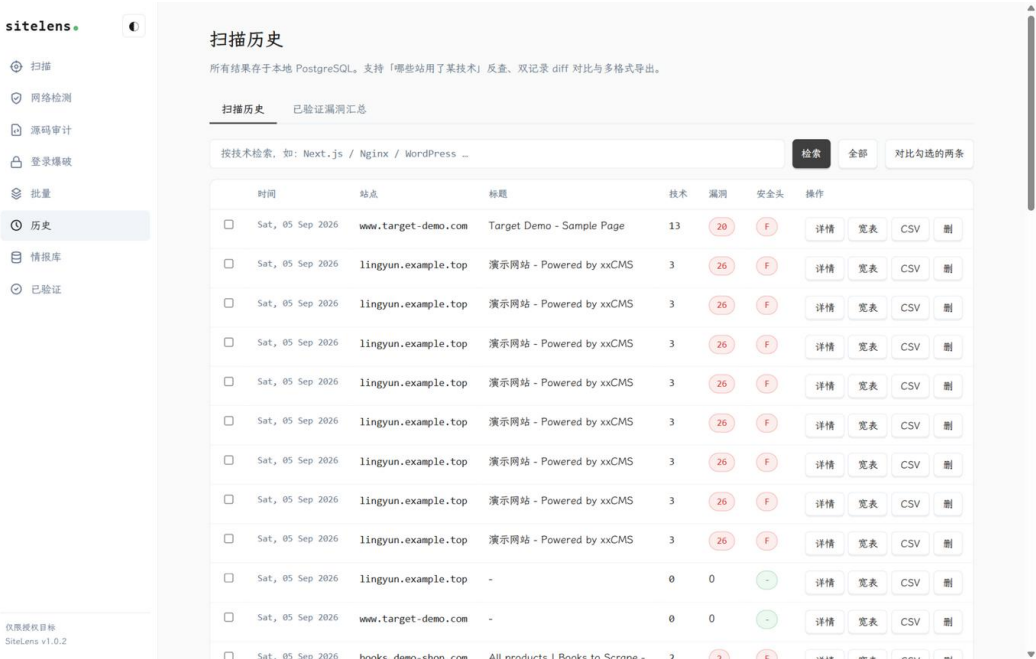


图 4-9 扫描历史与记录详情

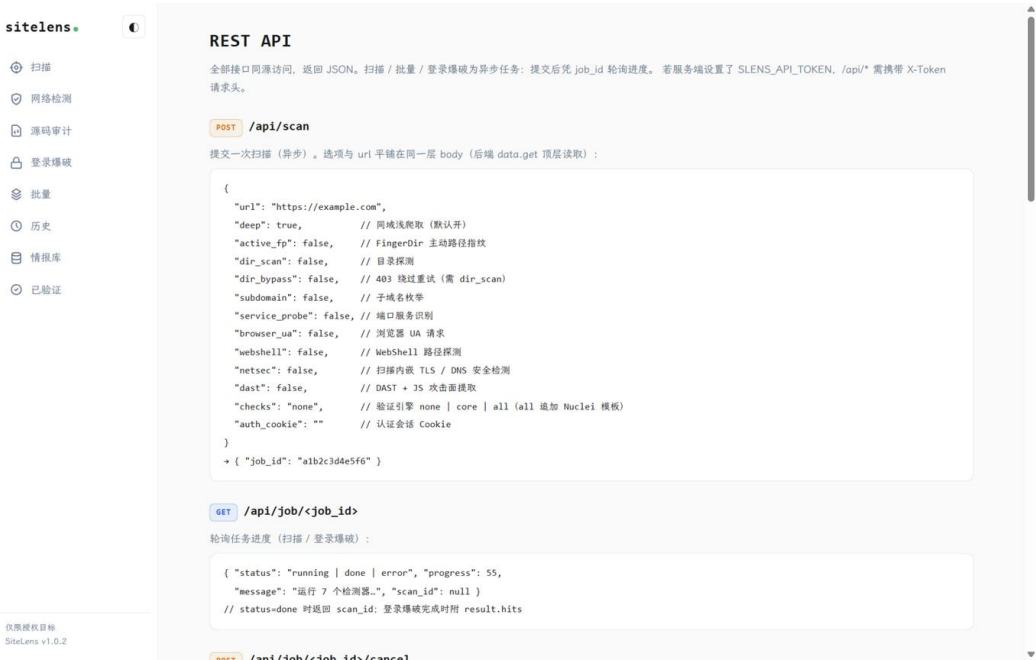


图 4-10 REST API 接口文档

5 系统测试

本章从测试目的、测试环境与测试用例三个方面对系统进行验证。

5.1 测试目的

测试目的：（1）验证各功能模块在正常输入下输出正确结果，异常输入下优雅报错而非崩溃；（2）验证面向对象设计的可扩展性（新增检测器是否零改引擎）；（3）验证合规与安全约束真实生效（内网目标必须被拒绝、限速必须生效）；（4）评估真实站点的识别效果与误报率。

测试采用三层结构：第一层为 49 项单元/集成测试（unittest 框架，全部离线可重复，覆盖校验器、解析器、匹配器、比较器等核心逻辑）；第二层为 32 项 API 冒烟测试（覆盖页面可达、参数校验、边界条件、四种导出格式与统计接口）；第三层为真实环境实测（text-well.com 等真实站点与自构造的漏洞样本页）。测试环境如表 5-1 所示。

表 5-1 测试环境

项目	配置
操作系统	Windows 10/11 x64
处理器 / 内存	x64 四核及以上 / 16GB
Python 环境	Python 3.10 + Flask 3 + requests + BeautifulSoup4 + psycpg2
数据库	PostgreSQL 18 (sitelens 库，含全部知识库数据)
浏览器	Firefox / Edge 最新版（前端验证）
测试靶标	text-well.com（真实站）、自构造泄露样本页、内网地址（负向用例）

5.2 测试用例

主要功能测试用例及结果如表 5-2 所示。

表 5-2 主要功能测试用例

编号	测试项	测试方法	预期结果	结论
T01	目标校验	提交 http://127.0.0.1 与内网 IP	拒绝扫描并提示不合法目标	通过
T02	异常输入	提交空 URL、ftp:// 协议	返回明确错误信息	通过
T03	指纹识别	扫描 text-well.com（深度模式）	识别出 13+ 项技术并按类别分组	通过

编号	测试项	测试方法	预期结果	结论
T04	版本捕获	对带版本头的站点扫描	正确捕获如 nginx/1.24.0 版本号	通过
T05	置信度合并	多来源命中的同一组件	合并为一条且置信度提升	通过
T06	验证引擎	对构造的 .git 泄露靶页扫描	命中「Git 仓库泄露」已验证漏洞	通过
T07	软 404 基线	对不存在路径批量探测	自定义 404 页不被误报	通过
T08	二次确认	模拟间歇性异常响应	单次命中不采信，两次命中才上报	通过
T09	情报三级判定	构造 Next.js 13.5.1 站点	输出确认受影响 CVE-2025-29927	通过
T10	区间排除	构造已修复版本站点	excluded 结论不输出	通过
T11	KEV 红标	检索含 KEV 漏洞的组件	命中项带在野利用红标	通过
T12	语义检索	情报库检索 grafana	亚秒返回语义相关结果	通过
T13	安全评分	扫描标准站点	A+ - F 评分与中文修复建议正确	通过
T14	网络层检测	对自有域名执行 TLS/SPF 检测	证书状态与 DNS 记录判定正确	通过
T15	源码审计	上传含危险调用的样本	污点分析标记高危数据流	通过
T16	登录爆破门禁	不勾选授权声明	无法启动爆破任务	通过
T17	批量与导出	批量 3 站点并导出宽表 CSV	进度正常，导出 78 列 CSV 正确	通过
T18	历史 diff	同站两次扫描做对比	正确输出新增/消失/版本变化	通过
T19	扫描历史检索	按技术名检索历史	返回全部使用该技术的站点	通过
T20	Token 鉴权	无 Token 调用受保护 API	401 拒绝	通过

测试结果汇总如表 5-3 所示。全部 49 项单元测试与 32 项冒烟测试通过；真实站点实测中，text-well.com 标准识别 5.7 秒检出 13 项技术（含通过内联脚本指纹检出的 Microsoft Clarity），与人工核对结果一致；自构造泄露样本 4 项全部命中，干净站点零误报。

表 5-3 测试结果汇总

测试层次	数量	结果
单元/集成测试（unittest）	49 项	全部通过

测试层次	数量	结果
API 冒烟测试（页面/参数/边界/导出）	32 项	全部通过
真实站点实测（text-well.com 标准识别）	13 项技术	识别正确，5.7s 完成
靶场回归（构造泄露样本）	4 项	全部命中，干净站零误报

测试中发现并修复的典型问题包括：扫描级扩展信息被子模块整体覆盖导致报告缺项、嵌于前端框架流式载荷中的内联脚本组件漏检（补充正文正则指纹解决）、中文 Windows 下 nslookup 输出编码不一致导致邮件安全检测误报（按 GBK 解码并逐行解析）、情报多来源合并时来源标记重复追加等，修复后均已完成回归验证。

6 总结

本实训完成了一个功能完整、可直接使用的验证型 Web 漏洞扫描器。系统的主要优点：（1）面向对象结构清晰，检测器继承体系使功能扩展成本极低，实训期间三次能力升级均未修改引擎代码；（2）「指纹 → 验证 → 情报」三级链路完整，三级判定机制把误报控制在了可用水平；（3）工程化程度高，具备异步任务、进度反馈、批量扫描、历史对比与多格式导出，不是演示玩具而是真实可用的工具；（4）合规设计内建于代码，SSRF 防护、限速与授权门禁不依赖使用者自觉。

系统也存在明显不足：（1）受 Python 线程与全局限速约束，吞吐量与 Go 系引擎存在量级差距，批量场景下偏慢；（2）依赖 requests 栈，无法覆盖需要浏览器渲染才能加载的组件，也缺少盲注类漏洞必需的 OOB 回连检测；（3）DAST 与污点分析为教学级深度，距工业工具仍有差距；（4）漏洞情报的时效性依赖定期运行更新脚本。

后续改进措施：优先为验证引擎增加响应内容抽取（extractor），把验证环节捕获的版本号回灌情报判定，进一步扩大「确认受影响」的覆盖面；其次按路径聚类合并模板请求以提升吞吐；中期引入轻量回连服务器以支持盲注检测；同时扩大靶场回归集，用可重复的断言持续校准误报率。

通过本次实训，我们完整经历了从需求分析、面向对象设计、编码实现到系统测试的工程全过程：封装让数据有了边界，继承与多态让能力可以生长，组合让系统可以像积木一样搭建；requests 与 BeautifulSoup 则是整个系统的眼睛。更重要的是，我们体会到安全工具「能力越大、克制越多」的合规责任——扫描器的每一行代码都

必须同时回答「能做什么」和「不做什么」。这些认识将伴随我们后续的专业学习与工程实践。